



Repetition Penalties

Repetition Penalties

1) What they are

Repetition penalties are sampling controls that discourage a model from reusing the same tokens it already produced (or that appear in the prompt). They reshape the next-token distribution so that repeated words/phrases become less likely, improving readability and reducing loops.

- **Frequency penalty** – Scales down tokens in proportion to **how many times** they've appeared.
- **Presence penalty** – Applies a **flat penalty** to any token that has appeared **at least once**.

Mental model: Presence = "don't repeat at all," Frequency = "if you repeat, do it sparingly."

2) How they work (intuitively)

At each generation step, the model has logits for all tokens. Penalties modify those logits **before** temperature/top-p/top-k are applied:

- **Frequency penalty:** $\text{new_logit} = \text{logit} - \alpha * \text{count}(\text{token})$
(Higher $\alpha \rightarrow$ stronger push toward new vocabulary.)
- **Presence penalty:** $\text{new_logit} = \text{logit} - \beta * \mathbb{I}(\text{token seen} \geq 1)$
(β applies once per token, regardless of count.)

Notes:

- "Token" $\neq$ word; subword tokenization means "cool" and "ing" might be separate tokens.

- Most APIs apply penalties to tokens seen in **prompt + generated text so far**.

3) When to use (and when not to)

Good fits

- **Long-form prose & marketing copy:** Reduce stale phrasing and echoing the prompt.
- **Brainstorming lists:** Encourage varied ideas and phrasing across bullets.
- **Safety/UX:** Mitigate degenerate loops ("the the the...") at higher temperatures.

Use cautiously or avoid

- **Code generation:** Repetition (brackets, keywords) is **required** → keep penalties near **0**.
- **Strict terminology** (legal, medical, translation): Over-penalizing can force awkward synonyms that change meaning.
- **Tables/structured JSON:** Repeated keys are expected; let structure—not penalties—govern repetition.

4) Practical defaults & tuning

Task Type	Frequency Penalty	Presence Penalty	Notes
Factual Q&A, summaries	0.0–0.3	0.0–0.2	Keep style tight; rely more on prompt structure.
Creative writing / marketing	0.3–0.8	0.1–0.5	Adds lexical variety; pair with moderate temperature.
Brainstorming lists	0.4–0.9	0.2–0.6	Helps ensure items don't sound the same.
Code, configs, schemas	0.0	0.0	Repetition is necessary for correctness.
Translation / legal	0.0–0.2	0.0–0.1	Preserve exact phrasing and terms of art.

Tip: If your provider allows negative values, negative penalties encourage repetition—rarely needed outside poetry forms or strict refrain patterns.

5) Interactions with other knobs

- **Temperature / top-p / top-k:** Penalties reshape logits **before** these apply. High temperature + high penalties can produce quirky synonyms—tune gradually.
- **Max tokens:** Penalties do **not** enforce concision; they only steer diversity.
- **Stop sequences:** Use for **clean boundaries**; penalties won't prevent overshooting the desired end.

6) Prompt patterns that pair well

Variety guardrails

Write 8 bullet points. Avoid reusing the same opening words across bullets. Vary verbs and sentence length.

Term consistency (when penalties are risky)

Use the exact term “payment authorization hold” throughout. Do not paraphrase this term.

Style + penalties

- Keep penalties modest and **ask explicitly** for variation: “use varied transitions,” “avoid repeating adjectives.”

7) Troubleshooting

Symptom	Likely Cause	Fix
Awkward synonyms / drift from terminology	Penalties too high for domain	Lower penalties; add a glossary/terminology constraint in prompt
Repetitive loops at high creativity	Penalties too low + high temperature	Raise frequency penalty to 0.3–0.6, or lower temperature

Symptom	Likely Cause	Fix
Broken code / malformed JSON	Penalties applied to structural tokens	Set penalties to 0; control with schema/JSON mode instead
Bullets sound the same	Penalties too low	Increase presence penalty slightly and instruct varied openings

8) Quick decision flow

1. **Is exact wording critical (code, legal, translation)?** → Set both penalties near **0**.
2. **Do you want more lexical variety (prose, marketing, ideas)?** → Start `freq ≈ 0.4`, `pres ≈ 0.2`.
3. **Seeing weird synonym swaps?** → Reduce penalties by 0.1–0.2 and/or lower temperature.
4. **Still repetitive at high creativity?** → Raise frequency penalty first (more controllable than presence).

9) Copy-paste presets

Balanced prose

```
temperature=0.6, top_p=0.9, frequency_penalty=0.4, presence_penalty=0.2
```

Idea generation

```
temperature=0.8, top_p=0.95, frequency_penalty=0.6, presence_penalty=0.3
```

Code / JSON

```
temperature=0.2, top_p=0.6, frequency_penalty=0.0, presence_penalty=0.0
```

Key takeaway

Repetition penalties are **diversity levers**, not length controls. Use **frequency** to meter how often tokens can recur and **presence** to nudge the model away from reusing anything it already said. Keep them low or off where **precision and structure** matter; turn them up when you need **variety and freshness**.